

## SINGAPORE-MIT ALLIANCE

Computational Engineering  
CME5232: Cluster and Grid Computing Technologies for Scientific Computing  
COMPUTATIONAL LAB No. 3  
20<sup>th</sup> July, 2007

### MPI IMPLEMENTATIONS ON ELEMENTARY ALGORITHMS

#### Objective

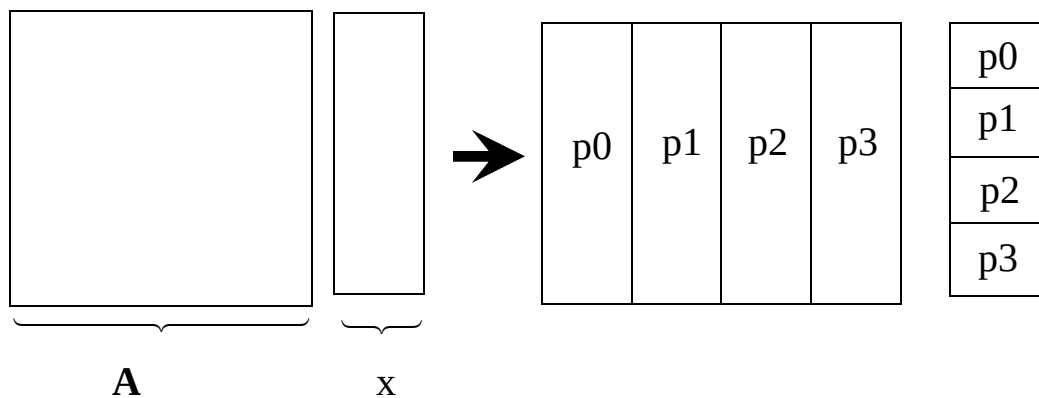
To experience MPI Library functions focusing on point to point and collective communications.

#### Background knowledge

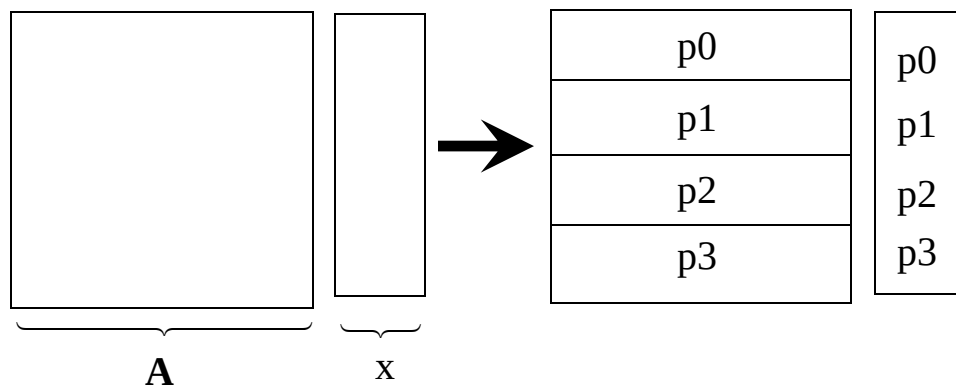
C/C++ languages and basic MPI functions

#### Matrix-Vector Multiplication

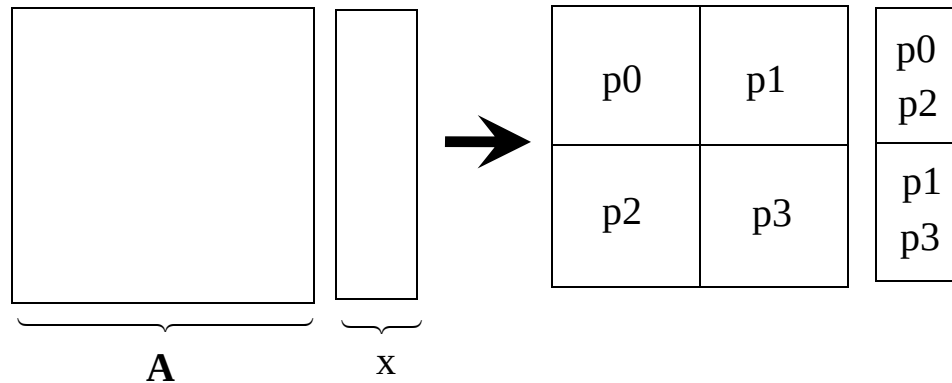
**Question 1:** Block-striped partitioning – column-wise block-striped partitioning



**Question 2:** Block-striped partitioning – row-wise block-striped partitioning



### Question 3: Block-checkerboard partitioning



Memory locations in C/C++?

Recall that **MPI\_Send**, **MPI\_Recv**, **MPI\_Bcast** and **MPI\_Reduce** all have a count and a data-type parameter. These two parameters allow the user to group data items having the same basic type into a single message. In order to use this, the grouped data items must be stored in contiguous memory locations.

To understand how data are stored in the memory locations by using C/C++ language, the following code has been generated as follows:

```
#include <stdio>
#include <stdlib>
#include <iostream>
using std::cout;
using std::endl;
void understandDoubleArray(int N);
void understandEquivalentSingleArray(int N);

int main(int argc, char **argv)
{
    int N = 4;
    understandDoubleArray(N);
    cout << endl;
    understandEquivalentSingleArray(N);
    return 0;
}

void understandDoubleArray(int N)
{
    int **A;
    int i, j;

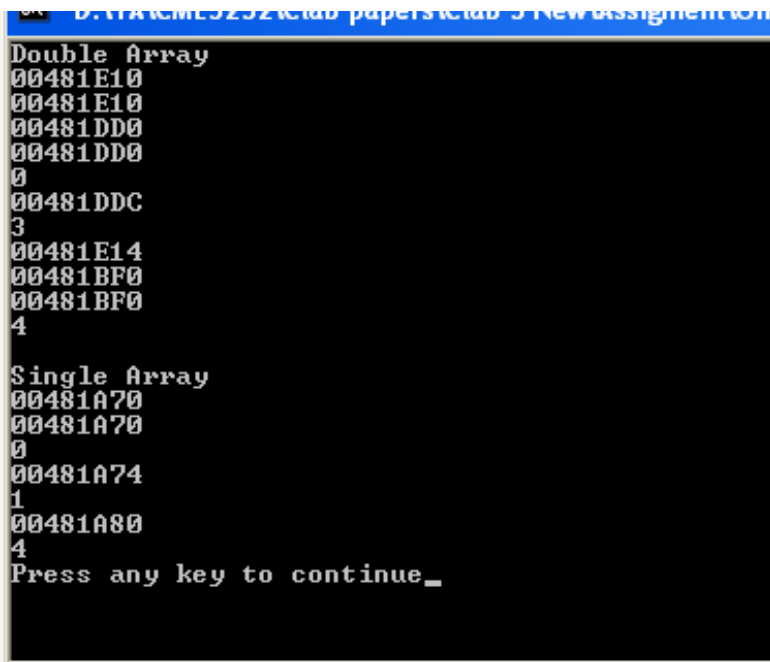
    A = new int*[N];
    for (i=0; i<N; i++)
        A[i] = new int[N];
    for (i=0; i<N; i++)
        for (j=0; j<N; j++)
            A[i][j] = i*N + j;
```

```

        cout << "Double Array" << endl;
        cout << A << endl;;
        cout << &(A[0]) << endl;
        cout << A[0] << endl;
        cout << &(A[0][0]) << endl;
        cout << A[0][0] << endl;
        cout << &(A[0][N-1]) << endl;
        cout << A[0][N-1] << endl;
        cout << &(A[1]) << endl;
        cout << A[1] << endl;
        cout << &(A[1][0]) << endl;
        cout << A[1][0] << endl;
    }
    void understandEquivalentSingleArray(int N)
    {
        int *A;
        int i, j;
        A = new int[N*N];
        for (i=0; i<N; i++)
            for (j=0; j<N; j++)
                A[i*N+j] = i*N + j;
        cout << "Single Array" << endl;
        cout << A << endl;
        cout << &(A[0]) << endl;
        cout << A[0] << endl;
        cout << &(A[1]) << endl;
        cout << A[1] << endl;
        cout << &(A[N]) << endl;
        cout << A[N] << endl;
    }
}

```

Result:



```

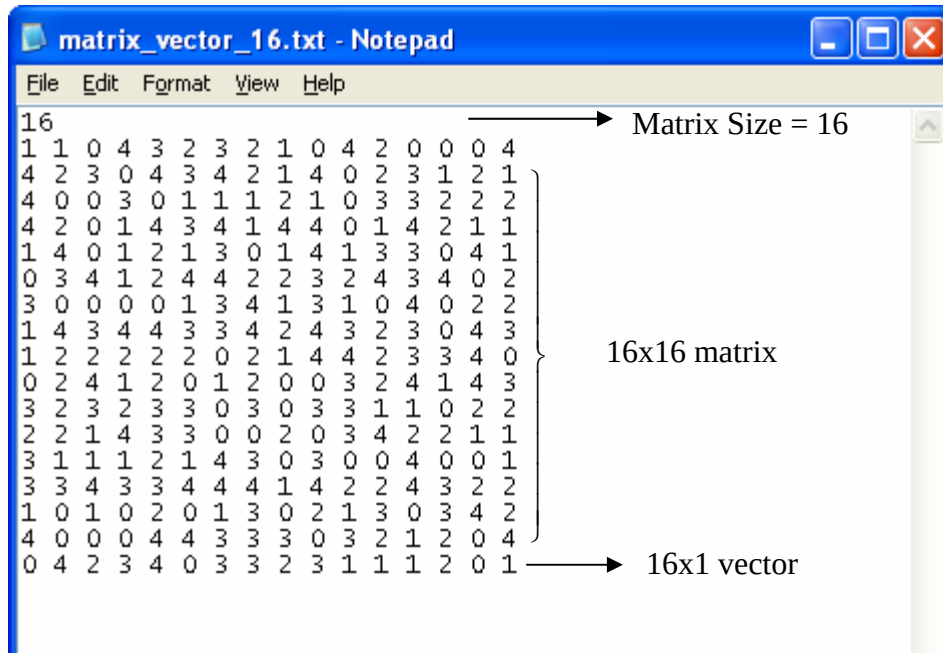
Double Array
00481E10
00481E10
00481DD0
00481DD0
0
00481DDC
3
00481E14
00481BF0
00481BF0
4

Single Array
00481A70
00481A70
0
00481A74
1
00481A80
4
Press any key to continue_

```

It can be seen from the result above that C/C++ guarantees that single-array elements are stored in contiguous memory locations whereas the double-array elements are not stored in contiguous memory locations.

What is the format of *matrix\_vector\_16.txt*?



**squareMatrixVectorGenerator.cpp** is provided for you to generate matrices and vectors of different sizes.

**Question 4:** Do what you are expected to do!

**Question 5:** Embarrassing matrix-matrix multiplication

Use of **squareMatrixMatrixGenerator.cpp** to generate the two square matrices?

**squareMatrixMatrixGenerator.cpp** is provided for you to generate square matrices **A** and **B** of the same size

```

$ gcc squareMatrixMatrixGenerator.cpp -o squareMatrixMatrixGenerator
$ ./squareMatrixMatrixGenerator
  
```

It will output a *matrix\_matrix\_<matrixSize>.txt* containing the matrices **A** and **B** information, and *matrix\_matrix\_<matrixSize>\_result.txt* containing the result of the matrix-matrix multiplication. Open both files to see what is inside the files.